

Cahier des charges — Module 114 : Codification / Chiffrement

Projet : Conversion de bases

Créé le 2025-09-08 — révisé le 2026-08-21

Consigne : l'utilisation de Copilot ou de tout autre outil d'IA générative est interdite pour ce projet.

Objectif du projet

Modifier le programme créé selon le premier cahier des charges (Morse). Le programme va évoluer en « Couteau Suisse » qui proposera divers outils liés aux notions vues en cours. La nouvelle « lame » ajoutée via ce cahier des charges concerne la conversion de bases.

Objectifs pédagogiques

Ce projet permet de :

- Manipuler l'arithmétique entière (division, modulo) dans une boucle
- Pratiquer la démarche de modélisation d'un algorithme avant de coder
- Comprendre le principe de position et de base numérique (décimal, binaire, octal)
- Concevoir « à la main » un algorithme de conversion, sans dépendre de fonctions prêtes du langage
- Valider une saisie utilisateur et gérer les cas d'erreur
- Vérifier son propre travail à l'aide de critères prédéfinis

Exemple d'exécution console

```
=== Couteau Suisse - Utilitaires ===
1. Convertir du texte en code Morse
2. Convertir des nombres entre différentes bases (Décimal <> Binaire <> Octal)
3. (à venir)
Veuillez entrer votre choix : 2

=== Convertisseur de bases ===
1. Décimal > Binaire
2. Binaire > Décimal
3. Binaire > Octal
4. Octal > Binaire
Veuillez entrer votre choix : 1
Entrez un nombre décimal : 42
Résultat : 101010
```

Fonctionnalités attendues

Fonction	Description
Menu principal	Proposer un menu permettant de choisir l'outil
Menu secondaire	Proposer un menu permettant de choisir le type de conversion à effectuer (6 entrées)
Saisie utilisateur	L'utilisateur entre un nombre à convertir (contrôler la validité)
Conversion	Effectuer la conversion selon l'option choisie
Affichage	Afficher clairement le résultat de la conversion
Gestion d'erreurs	Vérifier que l'entrée est correcte (ex. uniquement 0 et 1 pour le binaire) et redemander si invalide

ATTENTION : les conversions ne doivent pas utiliser de fonction « toute prête » de C# (par exemple `Convert.ToInt32(binaire, 2)` ou `Convert.ToString(nombre, 2)`). L'algorithme de conversion doit être écrit à la main.

1. Notions de base

- Qu'est-ce qu'une base numérique (décimal, binaire, octal) ?
- Pourquoi représenter un nombre dans une autre base ?
- Principe de la division/multiplication successive pour convertir

2. Concepts informatiques

- Variables (chaîne de caractères, entiers, tableau associatif / dictionnaire)
- Boucles (`while`, `do while`)
- Conditions (`if`, `else`)
- Fonctions simples (ex. : `ConvertDecimalToBinary(int decimalNumber)`)

3. Modélisation

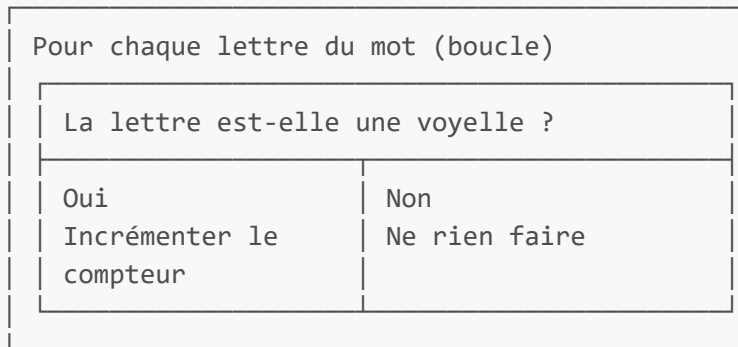
Avant d'écrire la moindre ligne de code, l'élève doit modéliser sa fonction principale de conversion. **Trois notations sont acceptées** — au choix de l'élève :

- **Structogramme (Nassi-Shneiderman)** : blocs imbriqués sans flèches, très lisible pour représenter séquence / condition / boucle.
- **Pseudocode structuré** : texte semi-formel, rapide à écrire, se transpose presque ligne à ligne en C#.
- **Organigramme (flowchart)** : boîtes reliées par des flèches, plus visuel.

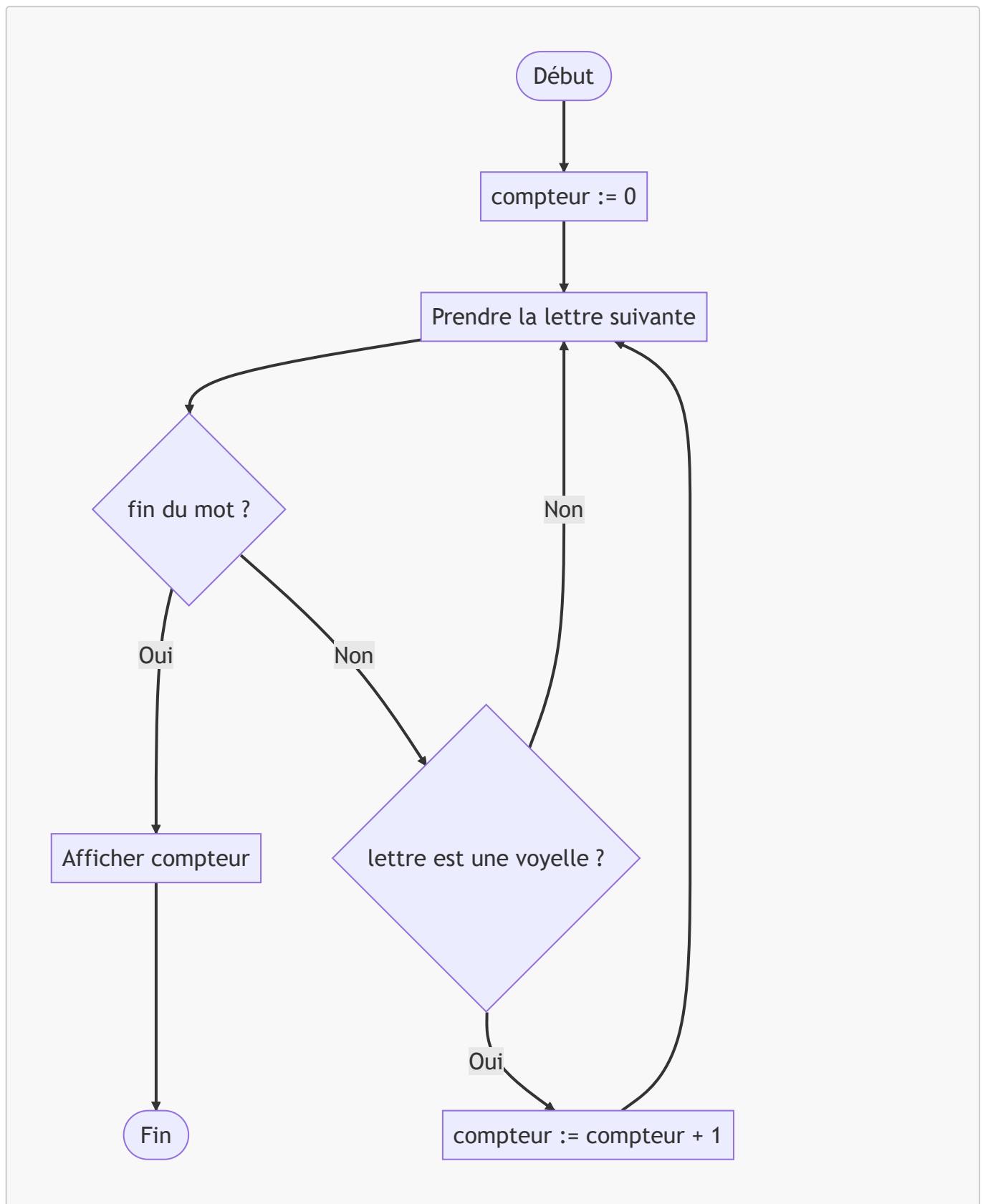
Le modèle doit être rendu **avant** le début du codage (feuille, photo, ou fichier séparé) et couvrir au minimum une fonction de conversion (ex. `ConvertDecimalToBinary`). Il n'y a pas de corrigé fourni ici : c'est précisément l'algorithme à trouver et à modéliser soi-même, avant de le coder.

Exemple — les 3 notations pour la même logique

Exemple générique (comptage de voyelles), sans lien avec la logique du projet, pour ne pas donner la solution.

Structogramme :**Pseudocode :**

```
FONCTION CompterVoyelles(mot)
    compteur ← 0
    POUR CHAQUE lettre DANS mot
        SI lettre EST DANS ("A", "E", "I", "O", "U", "Y") ALORS
            compteur ← compteur + 1
        FIN SI
    FIN POUR
    RETOURNER compteur
FIN FONCTION
```

Organigramme (flowchart) :

4. Structure du programme

1. Accueil et explication du programme
2. Choix de l'utilitaire
3. Affichage et explication du sous-programme
4. Modélisation (structogramme, pseudocode ou organigramme)

- 5. Conversion
- 6. Affichage du résultat

5. Validation — critères d'acceptation

Avant de considérer le projet terminé, l'élève vérifie que chacun des points suivants est rempli (première approche des tests d'acceptance) :

- ☐ Un modèle (structogramme, pseudocode ou organigramme) d'au moins une fonction de conversion a été produit avant le codage
- ☐ Le menu principal propose l'option « Convertir des nombres entre différentes bases »
- ☐ Le sous-menu propose les types de conversion attendus
- ☐ Convertir 42 (décimal) donne bien 101010 (binaire)
- ☐ Une entrée invalide (ex. « 2 » en binaire) est détectée et l'utilisateur peut ressaisir
- ☐ Aucune fonction de conversion « toute prête » du langage (`Convert.ToInt32`, etc.) n'est utilisée dans le code

Simplifications (en cas de blocage)

- Uniquement des entiers positifs, en décimal, binaire, octal

Volume horaire

Volume indicatif pour ce projet : **~8 périodes** (environ 2 à 3 semaines, à raison de 3 périodes/semaine).

Estimation ajustable selon le rythme de l'élève et en accord avec l'enseignant.

Outils et environnement

- **Langage** : C# console (application console, pas d'interface graphique)
- **IDE** : Visual Studio Code ou Visual Studio Community

Évolutions

- Ajout de la conversion hexadécimale
- Export du résultat dans un fichier texte
- Ajout de la conversion avec virgule (sur la base des exercices vus en cours)
- Paramètres de ligne de commande : effectuer une conversion directement au lancement (ex. `Bases.exe --from dec --to bin 42`), sans passer par le menu interactif
- Colorisation de la console : résultat en vert, messages d'erreur en rouge
- Mode DEBUG : afficher les étapes intermédiaires du calcul (ex. les divisions/restes successifs)
- Tests unitaires sur les fonctions de conversion (ex. avec xUnit)

Ces évolutions sont facultatives, à réaliser une fois les fonctionnalités de base validées — elles ne sont pas comptées dans le volume horaire indiqué ci-dessus.